

LAPORAN PRAKTIKUM

PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 4: Mengoperasikan Database dan Generate Data Otomatis dengan Laravel

Tugas Praktikum 1 - Pengembangan Aplikasi LaporanBanjir Berbasis Database

Disusun Oleh:

Nama : Mochammad Eyckies Bintang Satryaz

NIM : 707012500042

Kelas : D4SIKC-49-04

Mata Kuliah : PENGEMBANGAN APLIKASI BERBASIS WEB

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Adapun tujuan yang ingin dicapai dalam pengerjaan Tugas Praktikum 1 Modul 4 ini adalah:

- Menghubungkan project LaporanBanjir dari Modul 3 dengan database MySQL melalui konfigurasi pada file .env.
- Membuat dan menjalankan migration untuk membentuk tabel laporan sehingga data laporan tidak lagi disimpan dalam bentuk array pada Controller.
- Membuat model Laporan yang terhubung dengan tabel laporan menggunakan properti \$table dan \$fillable.
- Mengubah method penyimpanan pada Controller agar data dari form pelaporan disimpan secara permanen ke database menggunakan Laporan::create().
- Menampilkan kembali data laporan yang tersimpan di database pada halaman daftar laporan menggunakan Laporan::all() beserta penentuan status genangan (Waspada, Siaga, Awas).
- Menerapkan database seeding dengan library Faker (locale id_ID) untuk menghasilkan data laporan dummy secara otomatis sebagai bahan pengujian tampilan aplikasi.

2. Alat dan Bahan

Perangkat Keras

- Laptop/PC dengan sistem operasi Windows 11, prosesor setara 1 GHz ke atas, dan RAM minimal 8 GB.

Perangkat Lunak

- XAMPP (Apache & MySQL/MariaDB 10.4.32) sebagai server lokal serta phpMyAdmin untuk mengelola database.
- PHP 8.2.12 (CLI).
- Laravel Framework versi 12.69.3.
- Composer 2.10.3 sebagai dependency manager PHP.
- Visual Studio Code sebagai teks editor/IDE.
- Google Chrome sebagai browser untuk menjalankan dan menguji aplikasi.
- Git sebagai version control.

3. Langkah-Langkah Praktikum

Praktikum dikerjakan dengan mengembangkan kembali project LaporanBanjir dari Modul 3 pada folder Modul4-LaporanBanjir. Langkah-langkah berikut mencakup seluruh ketentuan Tugas Praktikum nomor 1, mulai dari konfigurasi database, migration, model, penyimpanan data dari form, penampilan data dari database, hingga pembuatan seeder dengan Faker.

a. Konfigurasi Database

1. Menjalankan Apache dan MySQL melalui XAMPP Control Panel sampai keduanya berstatus aktif.
2. Membuka phpMyAdmin pada <http://localhost/phpmyadmin>, lalu membuat database baru bernama db_laporbanjir.
3. Mengubah konfigurasi koneksi database pada file .env sehingga project LaporanBanjir terhubung ke MySQL, seperti potongan berikut:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laporbanjir
DB_USERNAME=root
DB_PASSWORD=
```

Nilai DB_CONNECTION diubah dari sqlite menjadi mysql dan DB_DATABASE diset ke db_laporbanjir.

b. Membuat Migration Tabel laporan

1. Membuat file migration untuk tabel laporan dengan perintah berikut:

```
php artisan make:migration create_laporan_table
```

2. Mengisi file database/migrations/2026_10_06_035357_create_laporan_table.php dengan struktur kolom sesuai ketentuan tugas, yaitu id, nama_pelapor, lokasi, tinggi_genangan (integer, cm), tanggal_kejadian (date), dan timestamps:

```
public function up(): void
{
    Schema::create('laporan', function (Blueprint $table) {
        $table->id();
        $table->string('nama_pelapor');
        $table->string('lokasi');
        $table->integer('tinggi_genangan');
        $table->date('tanggal_kejadian');
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('laporan');
}
```

3. Menjalankan migration sehingga tabel laporan terbentuk di database db_laporbanjir:

```
php artisan migrate
```

c. Membuat Model Laporan

1. Membuat model Laporan dengan perintah berikut:

```
php artisan make:model Laporan
```

2. Mengisi file app/Models/Laporan.php dengan properti \$table dan \$fillable agar model terhubung dengan tabel laporan dan kolomnya boleh diisi secara mass assignment:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Laporan extends Model
{
    protected $table = 'laporan';

    protected $fillable = [
        'nama_pelapor',
        'lokasi',
        'tinggi_genangan',
        'tanggal_kejadian',
    ];
}
```

d. Menyimpan Data dari Form ke Database

1. Mengubah method penyimpanan pada app/Http/Controllers/LaporBanjirController.php. Data yang dikirim dari form pelaporan divalidasi terlebih dahulu, lalu disimpan ke tabel laporan menggunakan Laporan::create():

```
<?php

namespace App\Http\Controllers;

use App\Models\Laporan;
use Illuminate\Http\Request;

class LaporBanjirController extends Controller
{
    public function form()
    {
        return view('lapor-banjir.form');
    }
}
```

```

public function kirim(Request $request)
{
    $validated = $request->validate([
        'nama_pelapor' => ['required', 'string', 'max:100'],
        'lokasi' => ['required', 'string', 'max:150'],
        'tinggi_genangan' => ['required', 'numeric', 'min:1'],
        'tanggal_kejadian' => ['required', 'date'],
    ]);

    Laporan::create($validated);

    return view('lapor-banjir.konfirmasi', $validated);
}

public function daftar()
{
    $laporan = Laporan::all();

    return view('lapor-banjir.daftar', compact('laporan'));
}
}

```

Setelah data tersimpan, method kirim() menampilkan halaman konfirmasi yang tetap menggunakan komponen <x-alert> seperti pada Modul 3.

e. Routing

1. Menambahkan route pada file routes/web.php untuk menampilkan form, menerima data POST dari form, dan menampilkan daftar laporan:

```

<?php

use App\Http\Controllers\LaporBanjirController;
use Illuminate\Support\Facades\Route;

Route::get('/', [LaporBanjirController::class, 'form']);

Route::get('/lapor-banjir', [LaporBanjirController::class, 'form'])->name('lapor-banjir.form');
Route::post('/lapor-banjir/kirim', [LaporBanjirController::class, 'kirim'])->name('lapor-banjir.kirim');
Route::get('/lapor-banjir/daftar', [LaporBanjirController::class, 'daftar'])->name('lapor-banjir.daftar');

```

f. Menampilkan Data dari Database

1. Halaman daftar laporan tidak lagi menggunakan array, melainkan mengambil data dari database menggunakan Laporan::all(). Data kemudian diulang dan dikirim ke partial laporan-card.blade.php:

```

{{-- resources/views/lapor-banjir/daftar.blade.php --}}

<section class="grid">

```

```

@forelse ($laporan as $item)
    @include('partials.laporan-card', ['laporan' => $item])
@empty
    <div class="panel">Belum ada laporan banjir.</div>
@endforelse
</section>

```

2. Partial laporan-card.blade.php tetap menentukan status genangan (Waspada, Siaga, Awas) berdasarkan tinggi genangan seperti pada Modul 3:

```

{{-- resources/views/partials/laporan-card.blade.php --}}
<article class="card">
    <div class="card-body">
        @if ($laporan->tinggi_genangan < 30)
            <span class="status waspada">Waspada</span>
        @elseif ($laporan->tinggi_genangan <= 70)
            <span class="status siaga">Siaga</span>
        @else
            <span class="status awas">Awat</span>
        @endif
        <h3>{{ $laporan->lokasi }}</h3>
        <p>Pelapor: <strong>{{ $laporan->nama_pelapor }}</strong></p>
        <p>Tinggi genangan: <strong>{{ $laporan->tinggi_genangan }} cm</strong></p>
        <p>Tanggal kejadian: <strong>{{ \Illuminate\Support\Carbon::parse($laporan->tanggal_kejadian)->translatedFormat('d F Y') }}</strong></p>
    </div>
</article>

```

Seluruh halaman tetap menggunakan layout layouts/app.blade.php sehingga tampilan form, konfirmasi, dan daftar laporan konsisten dengan Modul 3.

g. Seeder dan Faker

1. Membuat file seeder untuk menghasilkan data laporan dummy secara otomatis:

```
php artisan make:seeder LaporanSeeder
```

2. Mengisi database/seeders/LaporanSeeder.php dengan Faker ber-locale id_ID. Seeder menghasilkan 15 data laporan dengan nama pelapor acak, lokasi acak dari daftar kecamatan di Kabupaten Bandung, tinggi genangan 10-150 cm, dan tanggal kejadian dalam satu bulan terakhir:

```

<?php

namespace Database\Seeders;

use Faker\Factory as Faker;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class LaporanSeeder extends Seeder

```

```

{
    public function run(): void
    {
        $faker = Faker::create('id_ID');
        $kecamatan = ['Dayeuhkolot', 'Baleendah', 'Bojongsoang', 'Rancaekek',
'Majalaya'];

        for ($i = 0; $i < 15; $i++) {
            DB::table('laporan')->insert([
                'nama_pelapor' => $faker->name,
                'lokasi' => 'Kecamatan '.$faker->randomElement($kecamatan),
                'tinggi_genangan' => $faker->numberBetween(10, 150),
                'tanggal_kejadian' => $faker->dateTimeBetween('-1 month', 'now')->format('Y-m-d'),
                'created_at' => now(),
                'updated_at' => now(),
            ]);
        }
    }
}

```

3. Mendaftarkan LaporanSeeder pada file database/seeder/DatabaseSeeder.php melalui \$this->call([...]) agar dapat dijalankan sekaligus:

```

public function run(): void
{
    User::factory()->create([
        'name' => 'Test User',
        'email' => 'test@example.com',
    ]);

    $this->call([
        LaporanSeeder::class,
    ]);
}

```

4. Menjalankan seeder untuk mengisi tabel laporan dengan data dummy:

```

php artisan db:seed --class=LaporanSeeder
# atau menjalankan seluruh seeder
php artisan migrate:fresh --seed

```

h. Menjalankan Aplikasi dan Pengujian

1. Menjalankan server pengembangan Laravel dengan perintah:

```

php artisan serve

```

2. Membuka <http://127.0.0.1:8000/lapor-banjir>, mengisi form pelaporan, lalu menekan tombol Kirim Laporan.

3. Data dikirim ke route /lapor-banjir/kirim dan disimpan ke tabel laporan, kemudian muncul halaman konfirmasi dengan komponen <x-alert>.
4. Membuka <http://127.0.0.1:8000/lapor-banjir/daftar> untuk melihat daftar laporan yang berasal dari database beserta status genangan yang bervariasi.
5. Memeriksa phpMyAdmin untuk memastikan data dari form dan data hasil seeder benar-benar tersimpan pada tabel laporan di database db_laporbanjir.

4. Hasil dan Dokumentasi Program

Setelah seluruh tahapan dijalankan, aplikasi LaporanBanjir berhasil terhubung ke database MySQL db_laporbanjir. Alur form -> route -> controller -> model -> database berjalan dengan baik, sehingga laporan yang dikirim warga tersimpan secara permanen dan tidak lagi hilang ketika halaman ditutup. Tabel laporan memiliki kolom id, nama_pelapor, lokasi, tinggi_genangan, tanggal_kejadian, created_at, dan updated_at.

Halaman daftar laporan mengambil data langsung dari database menggunakan Laporan::all(). Data hasil seeder sebanyak 15 baris menampilkan status genangan yang bervariasi, yaitu Waspada untuk tinggi genangan di bawah 30 cm, Siaga untuk 30-70 cm, dan Awas untuk di atas 70 cm. Berikut dokumentasi hasil pengerjaan.

[Tempel screenshot di sini: Struktur tabel laporan di phpMyAdmin]

Gambar 4.1 Struktur tabel laporan pada phpMyAdmin.

[Tempel screenshot di sini: Halaman form pelaporan banjir]

Gambar 4.2 Halaman form pelaporan banjir.

[Tempel screenshot di sini: Halaman konfirmasi setelah laporan tersimpan]

Gambar 4.3 Halaman konfirmasi setelah laporan berhasil tersimpan ke database.

[Tempel screenshot di sini: Halaman daftar laporan dari database dengan status genangan bervariasi]

Gambar 4.4 Halaman daftar laporan yang datanya berasal dari database.

[Tempel screenshot di sini: Data hasil seeder pada tabel laporan di phpMyAdmin]

Gambar 4.5 Data hasil seeder pada tabel laporan di phpMyAdmin.

5. Kesimpulan

Berdasarkan pengerjaan Tugas Praktikum 1 Modul 4, dapat disimpulkan bahwa:

1. Project LaporBanjir berhasil dihubungkan dengan database MySQL db_laporbanjir melalui konfigurasi pada file .env tanpa mengubah kode program.
2. Migration memudahkan pembuatan dan pengelolaan struktur tabel laporan secara terstruktur dan dapat diulang, sehingga tabel beserta kolom-kolomnya terbentuk otomatis lewat perintah `php artisan migrate`.
3. Model Laporan dengan properti `$table` dan `$fillable` memungkinkan proses simpan dan ambil data dilakukan tanpa menulis perintah SQL secara manual.
4. Alur form, route, controller, model, dan view berjalan dengan baik sehingga data yang diinput melalui form tersimpan ke database dan dapat ditampilkan kembali pada halaman daftar laporan dengan status genangan yang sesuai.
5. Database seeding dengan library Faker ber-locale `id_ID` mampu menghasilkan 15 data laporan dummy secara otomatis, sehingga pengujian tampilan aplikasi dengan banyak data menjadi lebih cepat dan efisien.

6. Link GitHub

<https://github.com/kies-kis/PABW>